Inside a PDF

A structural guide for digital forensic examiners

A companion to PDF Structure Explorer. Edition 1, November 2026.

This PDF is also a teaching sample. It contains bookmarks, internal and external links, embedded font subsets, Info and XMP metadata, an attached file, object streams with a cross-reference stream, and an incremental update. Open it in the explorer to find each one.

Contents

How to use this guide 4

1. Reading a PDF 5

Two views of the same file 5

Where a reader starts 5

Bytes, characters and whitespace 5

2. Objects 7

The basic types 7

Strings and text encodings 7

Direct and indirect objects 8

Streams 8

Filters 8

3. File layout 10

Header 10

Cross-reference table 10

Trailer 10

How a reader opens a file 11

4. Compressed structure 12

Object streams 12

Cross-reference streams 12

Hybrid-reference files 12

5. Incremental updates and revisions 14

How an update works 14

Revisions 14

Why counting %%EOF markers misleads 14

6. The document 16

Document catalog 16

Page tree and inheritance 16

Page boxes and coordinates . . . 16
Resources . . . 16
Annotations . . . 17

7. Page content . . . 18
Content streams . . . 18
Paths . . . 18
Text . . . 18
Images and form XObjects . . . 18
Text you see and text you can extract . . . 19

8. Fonts . . . 20
Font dictionaries . . . 20
Composite fonts . . . 20
Subsets . . . 20

9. Metadata . . . 21
Document information dictionary . . . 21
XMP metadata . . . 21
Dates . . . 21
File identifiers . . . 22

10. Other features you will meet . . . 23
Linearization . . . 23
Encryption . . . 23
Embedded files . . . 23
Actions and JavaScript . . . 23
Interactive forms . . . 24
Digital signatures . . . 24

11. Anomalies and what they can indicate . . . 26

12. Reference . . . 28
PDF versions . . . 28
Where things are defined . . . 28
Sources and further reading . . . 30

How to use this guide

This guide explains how a PDF file is built: the bytes on disk, the objects those bytes describe, and how the objects fit together into pages, fonts, metadata and revisions. It is written for digital forensic examiners who need to understand what they are looking at when they open a PDF at the byte level, and it accompanies PDF Structure Explorer, which lets you select each part of a PDF and see it on the page and in the raw data at the same time.

The guide is educational, not a procedure. It describes what each structure is, why it exists, and what traces it can leave. Where a structure is relevant to an examination, a **Forensic note** explains why, together with the common innocent explanations. None of the features described here is, on its own, evidence of anything.

Each chapter ends with a **Try it** exercise that uses one of the explorer's built-in samples. The samples are small, purpose-built files, so the feature being discussed is always present and easy to find. This guide is also available as a PDF, and that PDF is itself one of the samples: it was built to contain bookmarks, links, embedded fonts, an attachment, compressed object streams and an incremental update, so you can explore the guide with the explorer.

Clause references point to both editions of the PDF standard: ISO 32000-1:2008, which defines PDF 1.7 and is available free from Adobe with the same page and clause numbers as the ISO edition, and ISO 32000-2:2020, which defines PDF 2.0 and is available free from the PDF Association after registration. A reference such as (ISO 32000-1 §7.5.4, page 40; ISO 32000-2 §7.5.4) gives the clause in each edition and the page in ISO 32000-1.

> **Note**
> The explorer reads the file in your browser and never changes it. Text-object boxes on the page are estimated from font widths, so they can be slightly off. The explorer is a learning aid, not a validator: it shows structure, and it flags a few common inconsistencies, but a file that looks clean in it has not been verified.

1. Reading a PDF

Two views of the same file

A PDF can be read in two ways, and an examiner needs both.

- **The physical view** is the sequence of bytes on disk: a header, then objects one after another, then cross-reference data, a trailer and an end-of-file marker. Every component has a byte offset, counted in decimal from zero at the first byte of the file.
- **The logical view** is a graph of objects that refer to one another. It starts at the trailer, which points to the document catalog, which points to the page tree, which points to each page, and so on. The order of objects in the file says nothing about their logical order: page 1 can be stored after page 50, and object 1 can appear anywhere.

The explorer shows both. The **Components** pane lists the logical structure grouped by role, the **Raw data** pane shows the physical bytes, and selecting a component in either view highlights it in the other and on the rendered page.

Where a reader starts

A PDF reader does not read a file from the beginning. It reads backwards from the end, looking for the keyword `startxref`, which is followed by the byte offset of the cross-reference data. The cross-reference data gives the byte offset of every object, and the trailer next to it names the document catalog. From there the reader follows references only as it needs them. This design lets a reader open a large file quickly, and it is also what makes incremental updates possible (see chapter 5).

Bytes, characters and whitespace

PDF syntax is written in ASCII, but a PDF is a binary file: streams can contain any byte values, and some parts of the file must not be altered by text-mode conversions. The syntax recognises six whitespace characters (NUL, TAB, LF, FF, CR and SPACE) and ten delimiters: `( ) < > [ ] { } / %`. Line endings may be CR, LF or CR LF, and a single file can mix them. Everything after a `%` up to the end of the line is a comment, except inside strings and streams (ISO 32000-1 §7.2.2, page 12; ISO 32000-2 §7.2.3).

Byte offsets in the cross-reference table are decimal, so the explorer's Text view numbers each line with its decimal offset. The Hex view uses hexadecimal offsets, as hex editors do. Both refer to the same bytes.

> **Forensic note**
> Offsets are exact. A single inserted or removed byte before an object moves every later offset, and a reader that finds an offset pointing at the wrong place must repair the file by scanning for objects. Different tools repair differently, so a damaged file can display differently in different readers.

Try it: Incremental update sample

Open the **Incremental update** sample and select **Header**. Note that the Raw data pane highlights bytes 0 to 14, and the details panel gives the same range. Switch to Hex view and find the four bytes above 127 on the second line: that is the binary marker described in chapter 3.

In the explorer, choose Samples, then Incremental update.

2. Objects

The basic types

Everything in a PDF is built from a small set of object types (ISO 32000-1 §7.3.1, page 13; ISO 32000-2 §7.3.1).

Type	Example	Notes
Boolean	`true` `false`	
Integer and real numbers	`42` `-3.5` `.25`	No exponent notation.
Literal string	`(Hello \(world\))`	Balanced parentheses allowed; backslash escapes.
Hexadecimal string	`<48656C6C6F>`	Two hex digits per byte; whitespace ignored.
Name	`/Type` `/A#20B`	Case-sensitive; `#xx` encodes a byte.
Array	`[1 0 R /Name (str)]`	Ordered, mixed types.
Dictionary	`<< /Type /Page /Rotate 90 >>`	Keys are names; a `null` value means the key is absent.
Stream	dictionary + `stream … endstream`	Always an indirect object.
Null	`null`	Also the value of a reference to a missing object.

By convention a dictionary's `/Type` entry says what kind of object it is, and `/Subtype` narrows it down: a font dictionary has `/Type /Font` and a `/Subtype` such as `/TrueType`. Readers usually know the type from context, so `/Type` is often optional and sometimes missing.

Strings and text encodings

A literal string uses backslash escapes for special characters (`\n`, `\(`, `\\`) and octal escapes for any byte (`\053`). A hexadecimal string spells out every byte in hex. Both forms can hold any bytes, so the same value can be written in several ways.

Strings meant to be read by people, such as metadata values, bookmarks and form field values, are *text strings* (ISO 32000-1 §7.9.2.2, page 86; ISO 32000-2 §7.9.2.2). They are encoded in one of three ways:

- **PDFDocEncoding**, a single-byte encoding similar to Latin-1.
- **UTF-16BE**, starting with the byte-order mark `FE FF`.
- **UTF-8**, starting with `EF BB BF` (PDF 2.0 only).

Strings inside page content are not text strings: their bytes are glyph codes whose meaning depends on the font (chapter 7).

Direct and indirect objects

An object written in place, inside another object, is *direct*. An object given an identity of its own is *indirect*: it is written as an object number, a generation number and the value between `obj` and `endobj`, and other objects refer to it by number (ISO 32000-1 §7.3.10, page 21; ISO 32000-2 §7.3.10).

```
12 0 obj
<< /Type /Font /Subtype /Type1 /BaseFont /Helvetica >>
endobj

… /Font << /F1 12 0 R >> …
```

The reference `12 0 R` means “object 12, generation 0”. In a newly created file every generation number is 0. A generation number only increases when an object number is freed and later reused, which is rare in practice, and objects stored in object streams always have generation 0. A reference to an object that does not exist is not an error: it is treated as `null`.

Streams

A stream is a dictionary followed by a block of bytes between the keywords `stream` and `endstream` (ISO 32000-1 §7.3.8, page 19; ISO 32000-2 §7.3.8). Streams hold anything potentially large: page content, images, embedded fonts, metadata and attachments.

```
4 0 obj
<< /Length 205 /Filter /FlateDecode >>
stream
…205 bytes of compressed data…
endstream
endobj
```

- `stream` must be followed by CR LF or LF (never CR alone); an end-of-line marker before `endstream` is expected but not counted in the length.
- `/Length` is the number of stored (encoded) bytes. It may be an indirect reference to an object written later, which lets software write the data before knowing its length.
- `/Filter` names one or more encodings applied to the data, and `/DecodeParms` gives their parameters.
- Every stream is an indirect object, and the dictionary itself is always direct.

The stream dictionary describes the bytes but not what they mean. The same kind of stream can be page content, an image or a font program; the object that refers to the stream decides how it is used.

Filters

Filters encode stream data (ISO 32000-1 §7.4, page 22; ISO 32000-2 §7.4). Several can be chained, and they are applied in the order listed when decoding.

Filter	Purpose
`FlateDecode`	zlib/deflate compression; by far the most common. Often used with a PNG predictor for cross-reference streams and images.

Filter	Purpose
`LZWDecode`	Older LZW compression.
`ASCIIHexDecode`, `ASCII85Decode`	Represent binary data as ASCII text.
`RunLengthDecode`	Simple run-length compression.
`DCTDecode`	JPEG image data.
`JPXDecode`	JPEG 2000 image data (PDF 1.5).
`JBIG2Decode`, `CCITTFaxDecode`	Bi-level (black and white) image compression.
`Crypt`	Per-stream encryption filter.

Forensic note

Most of a modern PDF is compressed, so searching the raw file for a word usually finds nothing, even when the word is visible on the page. Searches have to run against decoded streams, and text in page content may also need its font's ToUnicode map before it becomes readable (chapter 7). In the explorer, select a compressed stream and choose **Decoded** above the raw pane to see its decoded bytes.

Try it: Incremental update sample

In the **Incremental update** sample, select **1 0 obj** (the catalog, a dictionary) and then **4 0 obj** under **Page 1** (a content stream). Compare the details panel: only the stream reports stored bytes and a filter. Then select **2 0 obj** and look at **Refers to** and **Referred to by**: those are the indirect references that make up the object graph.

In the explorer, choose Samples, then Incremental update.

3. File layout

A classic PDF has four parts: a header, a body of objects, a cross-reference table and a trailer (ISO 32000-1 §7.5.1, page 38; ISO 32000-2 §7.5.1).

```
%PDF-1.7                        header
%âãÏÓ                           binary marker
1 0 obj … endobj                body: objects in any order
2 0 obj … endobj
…
xref                            cross-reference table
0 9
0000000000 65535 f
0000000015 00000 n
…
trailer                         trailer dictionary
<< /Size 9 /Root 1 0 R /Info 8 0 R /ID […] >>
startxref                       offset of the xref section
1730
%%EOF                           end-of-file marker
```

Header

The first line declares the version, for example `%PDF-1.7` (ISO 32000-1 §7.5.2, page 39; ISO 32000-2 §7.5.2). If the file contains binary data, the next line should be a comment containing at least four bytes above 127, so that transfer software treats the file as binary. From PDF 1.4 the catalog's `/Version` entry can declare a later version than the header, which lets an incremental update raise the version without rewriting the header (chapter 6).

Cross-reference table

The cross-reference table starts with the keyword `xref` and contains one or more subsections (ISO 32000-1 §7.5.4, page 40; ISO 32000-2 §7.5.4). Each subsection starts with the first object number and the number of entries, followed by one entry per object. Every entry is exactly 20 bytes:

```
nnnnnnnnnn ggggg n¶     in use: 10-digit byte offset, 5-digit generation
nnnnnnnnnn ggggg f¶     free: next free object number, generation
```

The final two bytes are an end-of-line pair (space and CR, space and LF, or CR and LF), which keeps every entry the same length so a reader can jump to any entry by arithmetic.

Free entries form a linked list that starts at object 0, which is always free with generation 65535. When an object is deleted, its entry becomes free and its generation number is incremented, so a later object reusing that number gets a new generation.

Trailer

The trailer dictionary follows the keyword `trailer` (ISO 32000-1 §7.5.5, page 42; ISO 32000-2 §7.5.5). Its most important entries are:

Key	Meaning
`/Size`	One more than the highest object number in the cross-reference data.
`/Root`	Reference to the document catalog.
`/Info`	Reference to the document information dictionary (optional).
`/ID`	Two byte strings identifying the file (chapter 9).
`/Prev`	Byte offset of the previous cross-reference section, in an updated file.
`/Encrypt`	Reference to the encryption dictionary, in an encrypted file.
`/XRefStm`	In a hybrid file, offset of a cross-reference stream (chapter 4).

The trailer is followed by `startxref`, the decimal byte offset of the cross-reference section, and the end-of-file marker `%%EOF`.

How a reader opens a file

1. Read the last part of the file and find `startxref` and its offset.
2. Load the cross-reference section at that offset and the trailer that goes with it.
3. If the trailer has `/Prev`, load the earlier section too, and repeat. Entries in later sections override earlier ones.
4. Follow `/Root` to the catalog, and load other objects only when they're needed.

Forensic note

Because readers use the cross-reference data instead of scanning, an object that no cross-reference entry points to is invisible to normal reading even though its bytes are in the file. Unreferenced objects are common and usually harmless (leftovers from editing software), but they are also where earlier content can survive. The explorer validates every in-use entry: select any entry under an xref table to see whether its offset really points at the start of that object.

Try it: Incremental update sample

In the **Incremental update** sample, expand **Revision 1** and then **xref table**. Expand a subsection and select an entry: the details panel confirms which object it points to. Then select **startxref** and use **Go to the cross-reference section** to follow the same path a reader takes.

In the explorer, choose Samples, then Incremental update.

4. Compressed structure

PDF 1.5 introduced two structures that make files smaller and also make them harder to read by eye.

Object streams

An object stream (`/Type /ObjStm`) is a stream that holds other objects (ISO 32000-1 §7.5.7, page 45; ISO 32000-2 §7.5.7). Its decoded data starts with pairs of numbers, object number and offset, one pair per contained object (`/N` pairs in all), followed by the objects themselves starting at offset `/First`. Contained objects are written without `obj` and `endobj`, and they have no byte offset of their own in the file.

Some objects can never be stored in an object stream: streams, objects with a non-zero generation number, the encryption dictionary, the object holding an object stream's `/Length`, and, in a linearized file, the catalog, the linearization dictionary and page objects.

Cross-reference streams

A cross-reference stream (`/Type /XRef`) replaces the cross-reference table and the trailer in one object (ISO 32000-1 §7.5.8, page 49; ISO 32000-2 §7.5.8). Its dictionary carries the trailer entries (`/Root`, `/Info`, `/ID`, `/Prev`), and its data is a binary table, usually Flate-compressed with a PNG predictor. `/W` gives the width in bytes of the three fields of each entry and `/Index` lists the object number ranges covered.

Type field	Meaning of the second and third fields
0	Free object: next free object number, generation.
1	Object at a byte offset: offset, generation.
2	Object in an object stream: object stream number, index within it.

Hybrid-reference files

A hybrid-reference file has a classic cross-reference table for PDF 1.4 readers plus a cross-reference stream, named by the trailer's `/XRefStm`, that newer readers also load (ISO 32000-1 §7.5.8.4, page 51; ISO 32000-2 §7.5.8.4). Objects stored in object streams are listed only in the stream, so an older reader sees fewer objects than a newer one.

> **Forensic note**
> In a file with object streams, most dictionaries, including the catalog and the document information dictionary, exist only in compressed form. A byte-level search for a metadata value or a keyword will not find them until the object streams are decoded. In the explorer, objects stored in an object stream are highlighted in the stream's **Decoded** data, and the details panel names the object stream that holds them.

Try it: Object streams sample

Open the **Object streams** sample. Under **File structure**, expand **Revision 1** and select the cross-reference stream: there is no `xref` keyword or `trailer` anywhere in the file. Select the catalog under **Document and metadata** and note that it is highlighted in decoded data, not in the file. Then select the object stream itself and look at **Objects stored inside**.

In the explorer, choose Samples, then Object streams.

5. Incremental updates and revisions

How an update works

A PDF can be changed without rewriting it. An incremental update appends to the end of the file (ISO 32000-1 §7.5.6, page 44; ISO 32000-2 §7.5.6):

- new objects and new versions of changed objects, reusing the same object numbers;
- a cross-reference section listing only those objects, plus free entries for any deleted objects;
- a trailer whose `/Prev` gives the offset of the previous cross-reference section;
- `startxref` and `%%EOF`.

The original bytes are left untouched. A reader starts from the last `startxref`, so it sees the newest version of every object and follows `/Prev` back for objects the update didn't change.

Revisions

Each `%%EOF` that ends an update marks the end of a complete revision. Every earlier revision is simply the start of the file up to its `%%EOF`, so it can be recovered exactly by cutting the file at that point. The explorer's revision switcher does this: it renders the page from the truncated file, lists only the objects that existed in that revision, and greys out the bytes that came later.

Objects replaced by a later revision stay in the file. The explorer lists them under **Replaced or deleted in a later revision**.

Why counting %%EOF markers misleads

Revisions should be found by following `startxref` and `/Prev`, not by searching for `%%EOF`:

- A **linearized** file has two `%%EOF` markers in a single revision (chapter 10).
- The text `%%EOF` can appear **inside a stream**, for example in an attached PDF.
- Some software writes **stray markers** or appends **data after the final** `%%EOF`.
- A file saved with **Save As** or **optimised** is usually rewritten in full, leaving a single revision with no history.

The second string in the trailer's `/ID` normally changes with each update while the first stays the same, and an update can raise the PDF version through the catalog's `/Version` (chapter 9).

> **Forensic note**
> Incremental updates are how signed documents remain verifiable after later changes (chapter 10), and they are how form filling, annotation and simple edits are saved by many applications. Earlier revisions can therefore contain content that is no longer visible: a changed title, an earlier version of a page's content stream, or a removed annotation. The absence of revisions proves nothing either way, because rewriting a file discards its history.

Try it: Incremental update sample

In the **Incremental update** sample, switch between **Revision 1** and **Revision 2** in the toolbar. Compare the rendered page, the **/Title** under the information dictionary, and the **Replaced or deleted** group. Select **Revision 2: trailer** and follow its `/Prev`.

In the explorer, choose Samples, then Incremental update.

6. The document

Document catalog

The catalog is the root of the object graph, named by the trailer's `/Root` (ISO 32000-1 §7.7.2, page 71; ISO 32000-2 §7.7.2). Entries an examiner often meets include:

Key	Purpose
`/Pages`	Root of the page tree (required).
`/Version`	A later PDF version than the header declares (PDF 1.4).
`/Metadata`	XMP metadata stream for the document (PDF 1.4).
`/Names`	Name trees, including `/EmbeddedFiles` and `/JavaScript`.
`/Outlines`	Bookmarks.
`/OpenAction`, `/AA`	Actions run when the document opens or on events.
`/AcroForm`	Interactive form fields.
`/StructTreeRoot`, `/MarkInfo`	Logical structure of a tagged PDF.

Page tree and inheritance

Pages are organised in a tree (ISO 32000-1 §7.7.3.2, page 76; ISO 32000-2 §7.7.3.2). Intermediate nodes (`/Type /Pages`) have `/Kids` and a `/Count` of the pages below them; leaves (`/Type /Page`) are pages (ISO 32000-1 §7.7.3.3, page 77; ISO 32000-2 §7.7.3.3). Page order is the order of the tree, not the order of object numbers or byte offsets.

Some page attributes are inherited from ancestor nodes when the page doesn't set them: `/Resources`, `/MediaBox`, `/CropBox` and `/Rotate` (ISO 32000-1 §7.7.3.4, page 80; ISO 32000-2 §7.7.3.4). A page that seems to have no resources may be using its parent's.

Page boxes and coordinates

`/MediaBox` is the full extent of the page. `/CropBox` is the region a viewer shows, and defaults to the media box. Three more boxes (`/BleedBox`, `/TrimBox`, `/ArtBox`) are used in print production. Page coordinates are in points (1/72 inch) with the origin at the bottom left and y increasing upwards (ISO 32000-1 §8.3.2, page 114; ISO 32000-2 §8.3.2).

Resources

Content streams refer to fonts, images and other resources by short names such as `/F1` or `/Im0`. The page's `/Resources` dictionary maps those names to objects (ISO 32000-1 §7.8.3, page 82; ISO 32000-2 §7.8.3), grouped by category: `/Font`, `/XObject`, `/ExtGState` (graphics state parameters), `/ColorSpace`, `/Pattern`, `/Shading` and `/Properties`.

Annotations

Annotations are objects placed on top of the page, listed in the page's `/Annots` array (ISO 32000-1 §12.5, page 381; ISO 32000-2 §12.5). Each has a `/Subtype` (for example `/Link`, `/Text`, `/Highlight`, `/Widget` for form fields, or `/FileAttachment`) and a `/Rect` giving its position. What an annotation displays comes from its appearance stream (`/AP`), which is separate from its data such as `/Contents`.

Forensic note

Content can exist without being visible: outside the crop box, under other content, drawn in the background colour, at zero size, in invisible text render mode, or in a page object that isn't part of the page tree. An annotation's appearance stream and its stored value can differ. None of these is unusual on its own: crop boxes are routinely used to trim pages, and scanned documents use invisible text for searchability.

Try it: Attachment and actions sample

Open the **Attachment and actions** sample. Select the catalog and look at **Refers to**: it points to the page tree, the name dictionary and an open action. Expand **Page 1** and select the annotations to see where each one sits on the page.

In the explorer, choose Samples, then Attachment and actions.

7. Page content

Content streams

A page's appearance is described by one or more content streams: a sequence of operators, each preceded by its operands (ISO 32000-1 §7.8.2, page 81; ISO 32000-2 §7.8.2). For example, `72 640 468 4 re f` means "add a rectangle at (72, 640), 468 wide and 4 high, then fill it". If `/Contents` is an array, the streams are joined in order as if they were one.

The current transformation matrix (CTM) maps content coordinates to the page. The `cm` operator changes it, and `q` and `Q` save and restore the graphics state, including the CTM (ISO 32000-1 §8.4.4, page 127; ISO 32000-2 §8.4.4).

Paths

Paths are built with `m` (move), `l` (line), `c`, `v` and `y` (Bézier curves), `h` (close) and `re` (rectangle), then painted with `S` (stroke), `f` or `F` (fill), `B` (fill and stroke), their variants, or `n` (end without painting) (ISO 32000-1 §8.5, page 131; ISO 32000-2 §8.5). `W` or `W*` before the painting operator makes the path a clipping path, which limits everything drawn afterwards until the graphics state is restored (ISO 32000-1 §8.5.4, page 137; ISO 32000-2 §8.5.4).

Text

Text is drawn between `BT` and `ET` (ISO 32000-1 §9.4, page 248; ISO 32000-2 §9.4). Inside a text object:

- `Tf` selects a font resource and size: `/F1 12 Tf`.
- `Td`, `TD`, `Tm` and `T*` position the text.
- `Tj`, `TJ`, `'` and `"` show strings. `TJ` takes an array mixing strings and numbers that adjust spacing.
- Text state operators set character spacing (`Tc`), word spacing (`Tw`), horizontal scaling (`Tz`), leading (`TL`), rise (`Ts`) and render mode (`Tr`) (ISO 32000-1 §9.3, page 243; ISO 32000-2 §9.3).

Render mode 3 draws text invisibly. It is used by OCR software to place a searchable text layer under a scanned image.

Images and form XObjects

An image is usually an XObject: a stream with `/Subtype /Image`, `/Width`, `/Height`, `/ColorSpace` and `/BitsPerComponent`, painted by the `Do` operator (ISO 32000-1 §8.9, page 203; ISO 32000-2 §8.9). An image always fills the unit square of the current CTM, so the `cm` before `Do` sets its size and position. Small images can be written inline in the content stream between `BI`, `ID` and `EI` (ISO 32000-1 §8.9.7, page 214; ISO 32000-2 §8.9.7).

A form XObject (`/Subtype /Form`) is a reusable group of drawing operators in its own stream, with a `/BBox` and an optional `/Matrix` (ISO 32000-1 §8.10, page 217; ISO 32000-2 §8.10). Logos, repeated headers and annotation appearances are often form XObjects.

Text you see and text you can extract

The bytes in a `Tj` string are glyph codes, not characters. Which glyph each code draws depends on the font and its encoding, and which Unicode character it represents depends on the font's encoding or its `/ToUnicode` map (ISO 32000-1 §9.10.3, page 293; ISO 32000-2 §9.10.3). As a result:

- Text can be visible but not extractable, for example when it has no ToUnicode map or is drawn as paths or an image.
- Text can be extractable but not visible, for example in render mode 3 or hidden under other content.
- Extracted text can differ from what is displayed, because the ToUnicode map can say anything.

Forensic note

A frequently reported problem is "redaction" done by drawing a black rectangle over text. The text operators remain in the content stream, so the text can still be selected, extracted and read in the raw data. True redaction removes the content. When examining a page, compare what is drawn with what the content stream contains.

Try it: This guide (PDF edition) sample

Open **This guide (PDF edition)** from the Samples menu. Expand **Page 1** and its content stream, then select a few text objects. Switch the raw pane to **Decoded** to read the operators, and find the `Tf` operator that selects each font.

In the explorer, choose Samples, then This guide (PDF edition).

8. Fonts

Font dictionaries

A font dictionary has `/Type /Font`, a `/Subtype` and a `/BaseFont` name (ISO 32000-1 §9.5, page 253; ISO 32000-2 §9.5). Simple fonts (`/Type1`, `/TrueType`, `/Type3`) use one byte per glyph code and give glyph widths in `/FirstChar` and `/Widths`. Their `/FontDescriptor` gives metrics and flags, and points to the embedded font program if there is one (ISO 32000-1 §9.8, page 281; ISO 32000-2 §9.8): `/FontFile` for Type 1, `/FontFile2` for TrueType, `/FontFile3` for CFF and OpenType.

Fourteen standard fonts (Helvetica, Times, Courier in four styles each, plus Symbol and ZapfDingbats) could historically be used without embedding, because every reader was expected to provide them.

Composite fonts

A composite font (`/Subtype /Type0`) supports large character sets. It has one descendant CIDFont, an `/Encoding` CMap that maps codes to character identifiers (often `/Identity-H`, two bytes per glyph), and widths in the CIDFont's `/W` array.

Subsets

Embedded fonts are usually subsets containing only the glyphs the document uses. A subset's name starts with six uppercase letters and a plus sign, such as `ABCDEF+Calibri` (ISO 32000-1 §9.6.4, page 258; ISO 32000-2 §9.9.2). The tag only has to be unique within the file, and different software generates it differently.

Forensic note

Fonts carry traces of the software that made the file: the subset tags, the choice of simple or composite fonts, the presence of ToUnicode maps, and the embedded font programs themselves. Text added later by a different application often uses a different font object, even when it looks the same, and a subset can lack glyphs that later edits needed. These observations describe the file's history; they do not by themselves establish when or why it was edited.

Try it: This guide (PDF edition) sample

In **This guide (PDF edition)**, open **Resources** and select the fonts. Note the six-letter subset tags, and expand a font to find its descriptor, its embedded font program and its ToUnicode map.

In the explorer, choose Samples, then This guide (PDF edition).

9. Metadata

Document information dictionary

The document information dictionary, named by the trailer's `/Info`, holds `/Title`, `/Author`, `/Subject`, `/Keywords`, `/Creator` (the application that created the original content), `/Producer` (the software that wrote the PDF), `/CreationDate` and `/ModDate` (ISO 32000-1 §14.3.3, page 549; ISO 32000-2 §14.3.3). Values are text strings (chapter 2). PDF 2.0 deprecates every key except `/CreationDate` and `/ModDate` in favour of XMP.

XMP metadata

XMP metadata is an XML packet in a stream with `/Type /Metadata /Subtype /XML`, named by the catalog's `/Metadata` (ISO 32000-1 §14.3.2, page 548; ISO 32000-2 §14.3.2). It is usually uncompressed so that tools can find and edit it in place, and it is usually followed by whitespace padding so it can grow without rewriting the file. Metadata streams can also be attached to pages, images and fonts.

Information dictionary	XMP property
`/Title`	`dc:title`
`/Author`	`dc:creator`
`/Subject`	`dc:description`
`/Keywords`	`pdf:Keywords`
`/Creator`	`xmp:CreatorTool`
`/Producer`	`pdf:Producer`
`/CreationDate`	`xmp:CreateDate`
`/ModDate`	`xmp:ModifyDate`

XMP also has properties with no Info equivalent, such as `xmp:MetadataDate` (when the metadata was last changed) and `xmpMM:DocumentID` and `xmpMM:InstanceID` (identifiers for the document and this version of it).

When the two sources disagree, the spec's rule compares dates: if the XMP date stamp is the same as or later than the information dictionary's `/ModDate`, the XMP is authoritative; otherwise the information dictionary is the more recent.

Dates

Information dictionary dates use the format `D:YYYYMMDDHHmmSSOHH'mm`, where `O` is `Z`, `+` or `-` (ISO 32000-1 §7.9.4, page 87; ISO 32000-2 §7.9.4). PDF 1.7 also puts an apostrophe after the minutes. Any part after the year may be omitted, and if the time zone is missing the time zone is unknown. XMP uses ISO 8601 instead: `2026-03-01T09:00:00Z`.

File identifiers

The trailer's `/ID` array holds two byte strings (ISO 32000-1 §14.4, page 551; ISO 32000-2 §14.4). The first is set when the file is created and should never change; the second should change whenever the file is updated. The spec suggests generating them by hashing information such as the time, the file's location and size, and its metadata, but software is free to use any method.

Forensic note

Every metadata value is ordinary data that any tool can write. Disagreement between the information dictionary and XMP is common and usually innocent: many applications update one and not the other. Dates without time zones, Producer values that don't match the Creator, and `/ID` strings that are equal or different all describe how the file was written, which is useful context, but each has many ordinary explanations.

Try it: Incremental update sample

In the **Incremental update** sample, expand the information dictionary and the XMP stream. Fields marked with a warning disagree; select **/Title** to see the comparison and which source the date rule favours. Then switch to **Revision 1**, where both sources agree.

In the explorer, choose Samples, then Incremental update.

10. Other features you will meet

Linearization

A linearized file, often called “fast web view”, places everything needed for the first page at the start of the file so it can be displayed while the rest downloads (ISO 32000-1 Annex F, page 675; ISO 32000-2 Annex F). Its first object is a linearization parameter dictionary (`/Linearized`, with the file length `/L`, the page count `/N` and other offsets), followed by a first-page cross-reference section and trailer, which ends in its own `%%EOF`. That is why a linearized file has two `%%EOF` markers in one revision. An incremental update keeps the old linearization data but makes it obsolete.

Try it with the **Linearized** sample: expand **Revision 1** and find both `%%EOF` markers and the dummy `startxref` value of 0. *(In the explorer, choose Samples, then Linearized.)*

Encryption

An encrypted file has an `/Encrypt` entry in its trailer (ISO 32000-1 §7.6, page 55; ISO 32000-2 §7.6). Strings and streams are encrypted; the structure (object numbers, dictionaries, cross-reference data) is not. With the standard security handler (ISO 32000-1 §7.6.3, page 58; ISO 32000-2 §7.6.4) a file can have a user password (needed to open it) and an owner password (needed to change permissions). An empty user password is common: the file opens without a prompt, and the permissions are advisory, enforced by the reader. Cross-reference streams are never encrypted, and XMP metadata can be left unencrypted.

Try it with the **Encrypted** sample: the dictionaries and cross-reference data are readable, but stream data is not, and the page still displays because the user password is empty. *(In the explorer, choose Samples, then Encrypted.)*

Embedded files

Files can be attached to a PDF in two main ways: through the `/EmbeddedFiles` name tree in the catalog’s `/Names`, or with a `/FileAttachment` annotation on a page (ISO 32000-1 §7.11.3, page 102; ISO 32000-2 §7.11.3). Both use a file specification dictionary with a file name (`/F`, `/UF`) and an `/EF` entry pointing to an embedded file stream (ISO 32000-1 §7.11.4, page 103; ISO 32000-2 §7.11.4). The stream’s `/Params` can record the size, creation and modification dates and an MD5 checksum, and `/Subtype` can give a MIME type.

Actions and JavaScript

Actions make something happen when the document opens (`/OpenAction`), when an event occurs (`/AA`, additional actions), or when a link or bookmark is used (ISO 32000-1 §12.6, page 414; ISO 32000-2 §12.6). The type is in `/S`: for example `/GoTo` (a destination in the document), `/URI` (a web link), `/Launch` (open a file or application) and `/JavaScript`, whose `/JS` entry holds the script (ISO 32000-1 §12.6.4.16, page 430; ISO 32000-2 §12.6.4.17). Readers differ widely in which actions they support and allow. The explorer only shows the bytes of an action; it never runs it.

Interactive forms

The catalog's `/AcroForm` lists form fields (ISO 32000-1 §12.7.2, page 431; ISO 32000-2 §12.7.3). Each field has a name, a value and usually a widget annotation that displays it through an appearance stream. Filling a form and saving is typically an incremental update. XFA, an XML-based form technology, is deprecated in PDF 2.0.

Digital signatures

A signature is stored in a signature field whose value is a signature dictionary (`/Type /Sig`) (ISO 32000-1 §12.7.4.5, page 446; ISO 32000-2 §12.7.5.5) (ISO 32000-1 §12.8, page 466; ISO 32000-2 §12.8). Its key entries are:

Key	Meaning
`/Filter`, `/SubFilter`	The handler and format, for example `/adbe.pkcs7.detached` or `/ETSI.CAdES.detached`.
`/ByteRange`	Four numbers: two byte ranges of the file that are signed.
`/Contents`	The signature itself (a CMS/PKCS #7 object) as a hexadecimal string.
`/M`, `/Name`, `/Reason`, `/Location`	Optional signing time and details. They are covered by the signature, but the time is the signer's own claim, not a trusted timestamp.

`/ByteRange` covers the whole file as it was when signed, except the `/Contents` hex string itself, which can't sign itself. The second range ends at the end of that revision. Later incremental updates, such as filling a form or adding another signature, fall outside the signed ranges and leave the signature mathematically valid. Validating software then has to decide whether those later changes were permitted, for example by the document's modification permissions.

Forensic note

For any signature, compare where its byte range ends with where the file ends. If they differ, the file was changed after signing; the explorer's revision switcher shows exactly what the later revisions contain. Whether a change is acceptable depends on the signature's permissions and the validator's policy, not on the PDF structure alone. Checking the signature cryptographically requires a dedicated validator; the explorer only shows the structure.

Try it: Signed sample

Open the **Signed** sample and select the signature dictionary. The details panel lists the signed byte ranges and the excluded `/Contents`; expand it to highlight each range. Then compare the end of the signed range with **Revision 2**, which was added after signing.

In the explorer, choose Samples, then Signed.

Try it: Attachment and actions sample

Open the **Attachment and actions** sample. Find the embedded file stream and its `/Params`, the file attachment annotation on the page, and the open action with its JavaScript. Use **Decoded** to read the attached file's contents.

In the explorer, choose Samples, then Attachment and actions.

11. Anomalies and what they can indicate

The structures below are worth noticing because they reveal something about how a file was made or changed. Each has common innocent causes, and none is conclusive on its own. Where the explorer shows the anomaly, the last column says where to look.

Observation	What it is	Common innocent causes	In the explorer
Data after the final `%%EOF`	Bytes that no reader uses.	Padding by transfer or storage software; damaged downloads; some generators.	**Data after %%EOF** under the last revision.
More than one revision	Incremental updates.	Form filling, annotations, signatures, simple edits.	Revision switcher; **Replaced or deleted** group.
Cross-reference offset doesn't match	An entry points at the wrong byte.	Bugs in writing software; text-mode transfers that changed line endings.	Warning on the xref table and its entries.
`/Length` doesn't match the data	The stream's stated size is wrong.	Generator bugs; line-ending conversion.	Note in the stream's details.
Unreferenced objects	Objects no cross-reference entry or reference reaches.	Leftovers from editing; unused resources.	**Other objects**, and objects with no **Referred to by**.
Objects replaced in later revisions	Earlier versions of changed objects.	Any incremental update.	**Replaced or deleted** group.
Features newer than the declared version	For example object streams in a "PDF 1.4" file.	Very common: many writers don't update the header.	Warnings in the glossary.
Information dictionary and XMP disagree	Two metadata sources out of step.	Applications that update only one source.	Warning icons on metadata fields.
Non-zero generation numbers	An object number was reused.	Rare but legitimate after deletions.	The object label (`n g obj`).
Invisible or hidden content	Content not shown on the rendered page.	Crop boxes, OCR text layers, layout leftovers.	Compare boxes on the page with the content stream.
Signature range ends before the file does	Changes after signing.	Form filling, further signatures, annotations.	The signature's details and the revision switcher.
Encryption with an empty user password	Opens without a password; permissions are advisory.	Very common for "protected" documents.	**Encryption dictionary** under Document and metadata.

Try it: Anomalies sample

Open the **Anomalies** sample. It contains data after the final `%%EOF`, a stray `%%EOF`, a wrong cross-reference offset, a `/Length` that doesn't match, an unreferenced object and a version mismatch. Find each one using the table above.

In the explorer, choose Samples, then Anomalies.

12. Reference

PDF versions

Version	Reference published	Notable additions
PDF 1.0	1993	First public specification.
PDF 1.1	1996	Encryption.
PDF 1.2	1996	Interactive forms; linearization; ToUnicode maps; composite (Type 0) fonts.
PDF 1.3	2000	Digital signatures; logical structure; smooth shading; JavaScript actions.
PDF 1.4	2001	Transparency; XMP metadata streams; tagged PDF; catalog `/Version`.
PDF 1.5	2003	Object streams; cross-reference streams; JPEG 2000; optional content.
PDF 1.6	2004	AES encryption; OpenType font embedding.
PDF 1.7	2006	Standardised as ISO 32000-1:2008.
PDF 2.0	2017	ISO 32000-2 (dated revision 2020): AES-256; UTF-8 text strings; associated files; deprecation of most information dictionary keys and of XFA.

Where things are defined

Topic	ISO 32000-1:2008 (PDF 1.7)	ISO 32000-2:2020 (PDF 2.0)
Character set and whitespace	§7.2.2, page 12	§7.2.3
Object types	§7.3.1, page 13	§7.3.1
Stream objects	§7.3.8, page 19	§7.3.8
Indirect objects	§7.3.10, page 21	§7.3.10
Filters	§7.4, page 22	§7.4
File structure	§7.5.1, page 38	§7.5.1
File header	§7.5.2, page 39	§7.5.2

Topic	ISO 32000-1:2008 (PDF 1.7)	ISO 32000-2:2020 (PDF 2.0)
Cross-reference table	§7.5.4, page 40	§7.5.4
File trailer	§7.5.5, page 42	§7.5.5
Incremental updates	§7.5.6, page 44	§7.5.6
Object streams	§7.5.7, page 45	§7.5.7
Cross-reference streams	§7.5.8, page 49	§7.5.8
Hybrid-reference files	§7.5.8.4, page 51	§7.5.8.4
Encryption	§7.6, page 55	§7.6
Standard security handler	§7.6.3, page 58	§7.6.4
Document catalog	§7.7.2, page 71	§7.7.2
Page tree nodes	§7.7.3.2, page 76	§7.7.3.2
Page objects	§7.7.3.3, page 77	§7.7.3.3
Inheritance of page attributes	§7.7.3.4, page 80	§7.7.3.4
Content streams	§7.8.2, page 81	§7.8.2
Resource dictionaries	§7.8.3, page 82	§7.8.3
Text strings	§7.9.2.2, page 86	§7.9.2.2
Dates	§7.9.4, page 87	§7.9.4
File specification dictionaries	§7.11.3, page 102	§7.11.3
Embedded file streams	§7.11.4, page 103	§7.11.4
Coordinate spaces	§8.3.2, page 114	§8.3.2
Graphics state operators	§8.4.4, page 127	§8.4.4
Path construction and painting	§8.5, page 131	§8.5
Clipping paths	§8.5.4, page 137	§8.5.4
Images	§8.9, page 203	§8.9
Inline images	§8.9.7, page 214	§8.9.7
Form XObjects	§8.10, page 217	§8.10
Text state	§9.3, page 243	§9.3
Text objects	§9.4, page 248	§9.4
Font data structures	§9.5, page 253	§9.5
Font subsets	§9.6.4, page 258	§9.9.2
Font descriptors	§9.8, page 281	§9.8
ToUnicode CMaps	§9.10.3, page 293	§9.10.3

Topic	ISO 32000-1:2008 (PDF 1.7)	ISO 32000-2:2020 (PDF 2.0)
Annotations	§12.5, page 381	§12.5
Actions	§12.6, page 414	§12.6
JavaScript (ECMAScript) actions	§12.6.4.16, page 430	§12.6.4.17
Interactive form dictionary	§12.7.2, page 431	§12.7.3
Signature fields	§12.7.4.5, page 446	§12.7.5.5
Digital signatures	§12.8, page 466	§12.8
Metadata streams	§14.3.2, page 548	§14.3.2
Document information dictionary	§14.3.3, page 549	§14.3.3
File identifiers	§14.4, page 551	§14.4
Linearized PDF	Annex F, page 675	Annex F

Sources and further reading

- **ISO 32000-1:2008 (PDF 1.7)**: free copy hosted by Adobe, with the same page and clause numbers as the ISO edition: https://opensource.adobe.com/dc-acrobat-sdk-docs/pdfstandards/PDF32000_2008.pdf
- **ISO 32000-2:2020 (PDF 2.0)**: free from the PDF Association after registration: https://pdfa.org/sponsored-standards
- **PDF specification archive**: links to every version from PDF 1.0 to 2.0: https://pdfa.org/resource/pdf-specification-index/
- **Arlington PDF Model**: a machine-readable model of every PDF object and key, with the version each was introduced in: https://github.com/pdf-association/arlington-pdf-model